Physics-informed Machine Learning

Wireless Communication and Information Systems Laboratory (WCISL)
Department of Electrical and Computer Engineering, SNU

WCISL Wireless Communication and Information Systems Laboratory

Contents

I. Introduction to PIML

II. Physics-informed Neural Networks (PINN)

III. Physics-informed DeepONet (PIDON)

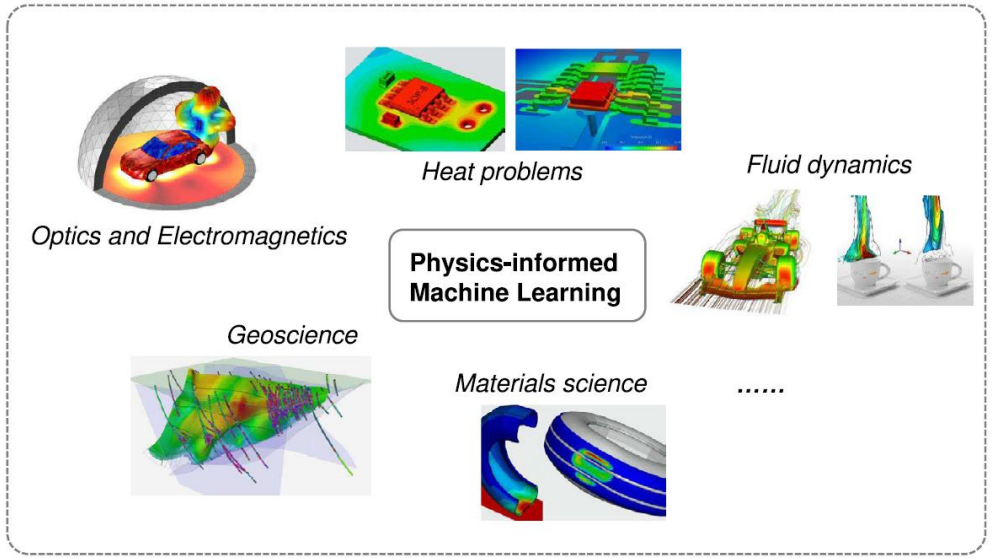
IV. Physics-informed Neural Operator (PINO)

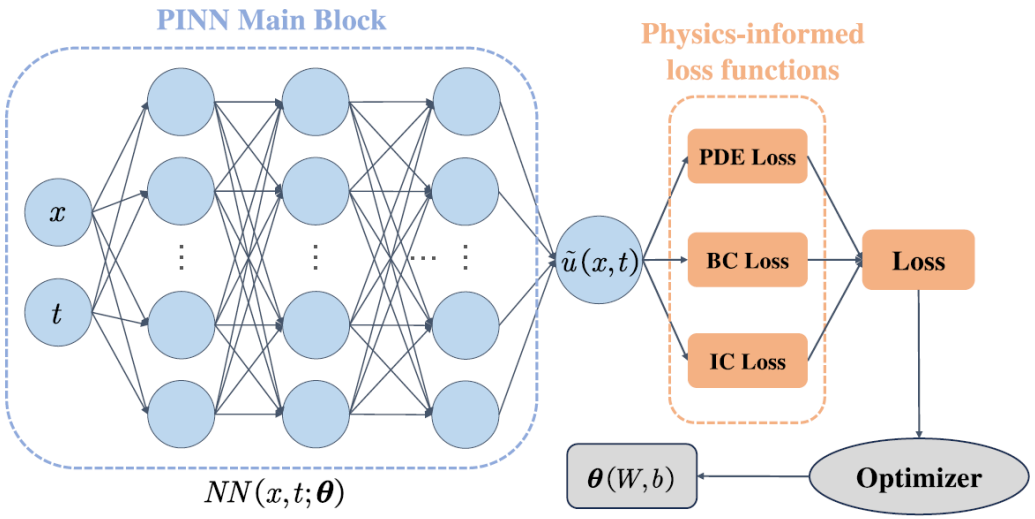
V. Related Works

VI. Conclusion

Introduction to PIML

- A deep learning approach that integrates physical laws into the loss function, enabling the discovery of physically consistent and reliable solutions even with limited data.
  - Many physical systems are governed by PDEs.
  - Classical solvers are accurate but computationally expensive for repeated simulations.
  - Physics-informed learning aims to combine data efficiency, physical consistency, and fast inference.





[1] Karniadakis, George Em, et al. "Physics-informed machine learning." Nature Reviews Physics 3.6 (2021): 422-440.

Physics-informed Neural Networks (PINN)

- Neural networks trained for supervised tasks while enforcing physical laws governed by nonlinear PDEs [2]
- Parametrized and nonlinear PDE of the general form: $u_t + \mathcal{N}[u;\lambda] = 0,\ x \in \Omega,\ t \in [0,T]$
  - $u(t,x)$ denotes the latent (hidden) solution.
  - $\mathcal{N}[\,\cdot\,;\lambda]$ is a nonlinear differential operator parametrized by λ.
- Two main classes of problems
  1. Data-driven solutions of PDE (forward problem)
     - Given fixed model parameters λ, what can be said about the unknown hidden state $u(t,x)$ of the system?
  2. Data-driven discovery of PDE (inverse problem)
     - What are the parameters λ that best describe the observed data?

[2] Raissi, Maziar, Paris Perdikaris, and George E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations." Journal of Computational physics 378 (2019): 686-707.

Physics-informed Neural Networks (PINN)

- Forward problem
  - $u_t + \mathcal{N}[u] = 0\ ,\ x \in \Omega,\ t \in [0,T]\ \rightarrow\ f \coloneqq u_t + \mathcal{N}[u]$
  - Approximate $u(t,x)$ by a DNN
  - Physics-informed residual network $f(t,x)$ sharing the same network parameters can also be constructed.
- Loss function design
  - $MSE = MSE_u + MSE_f$
    - $MSE_u = \frac{1}{N_u}\sum_{i=1}^{N_u}\left|u(t_u^i, x_u^i) - u^i\right|^2$, where $\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$ denote the IBC training data
    - $MSE_f = \frac{1}{N_f}\sum_{i=1}^{N_f}\left|f(t_f^i, x_f^i)\right|^2$, where $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ specify the collocations points for f
  - By designing a custom loss aligned with the PDE operator, physical structure is directly embedded into the network.
  - This enables learning with limited data, with the structure acting as an implicit regularizer.

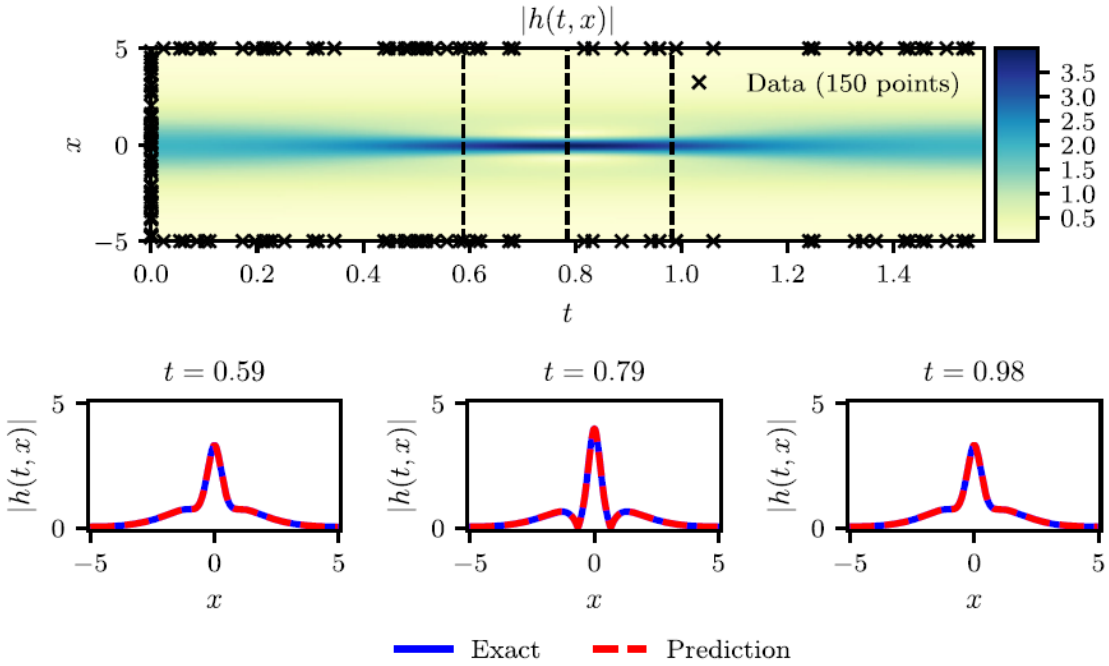
Physics-informed Neural Networks (PINN)

- Forward example: 1D nonlinear Schrödinger equation
  - $ih_t + 0.5h_{xx} + |h|^2 h = 0, \quad x \in [-5, 5], \; t \in [0, \pi/2]$
  - $h(0, x) = 2\,\mathrm{sech}(x)\,, \; h(t, -5) = h(t, 5), \; h_x(t, -5) = h_x(t, 5)$
- PINN design
  - $f := ih_t + 0.5h_{xx} + |h|^2 h$
  - $MSE = MSE_f + MSE_0 + MSE_b$

$$= \frac{1}{N_f}\sum_{i=1}^{N_f}\left|f\left(t_f^i, x_f^i\right)\right|^2 + \frac{1}{N_0}\sum_{i=1}^{N_0}\left|h\left(0, x_0^i\right) - h_0^i\right|^2$$

$$+ \frac{1}{N_b}\sum_{i=1}^{N_b}\left(\left|h^i\left(t_b^i, -5\right) - h^i\left(t_b^i, 5\right)\right|^2 + \left|h_x^i\left(t_b^i, -5\right) - h_x^i\left(t_b^i, 5\right)\right|^2\right)$$

  - $N_0 = 50, \; N_b = 50, \; N_f = 20000$
  - 5 hidden layers × 100 neurons w/ tanh activation

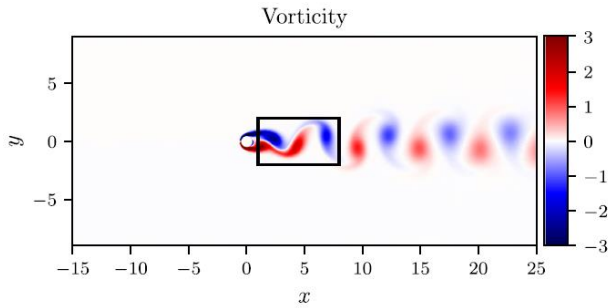


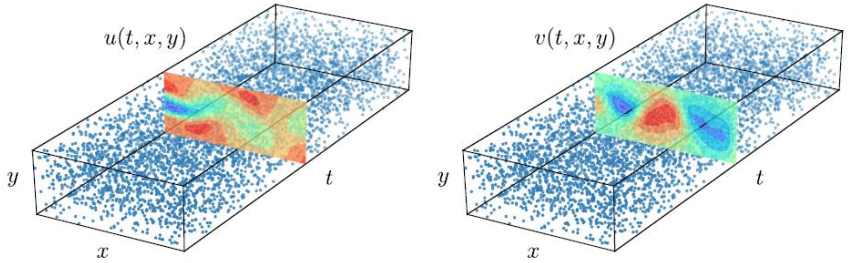
Physics-informed Neural Networks (PINN)

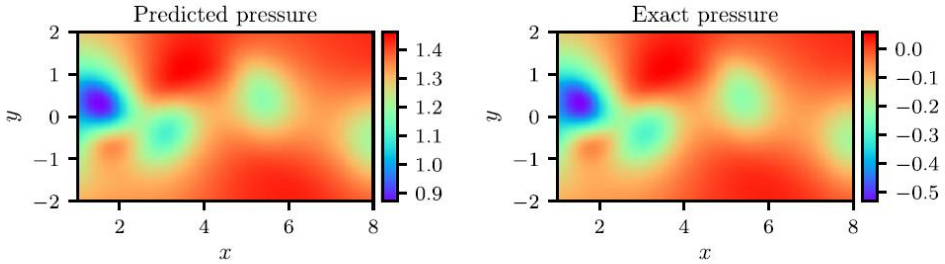
- Inverse problem
  - $u_t + \mathcal{N}[u;\lambda] = 0\ ,\ x \in \Omega,\ t \in [0,T]\ \rightarrow\ f \coloneqq u_t + \mathcal{N}[u;\lambda]$
  - Parameters of the differential operator λ turn into parameters of the $f(t,x)$
- Inverse example: 2D Navier-Stokes equation
  - $\begin{cases} u_t + \lambda_1(uu_x + vu_y) = -p_x + \lambda_2(u_{xx} + u_{yy}) \\ v_t + \lambda_1(uv_x + vv_y) = -p_y + \lambda_2(v_{xx} + v_{yy}) \end{cases}$ where $u(t,x,y)$, $v(t,x,y)$ denotes the velocity, $p(t,x,y)$ is the pressure
  - Stream function ψ ($u = \psi_y,\ v = -\psi_x$) satisfies the incompressible condition $u_x + v_y = 0$
  - $f \coloneqq u_t + \lambda_1(uu_x + vu_y) + p_x - \lambda_2(u_{xx} + u_{yy})$
  - $g \coloneqq v_t + \lambda_1(uv_x + vv_y) + p_y - \lambda_2(v_{xx} + v_{yy})$
  - $MSE = \frac{1}{N}\sum_{i=1}^{N}\left(\left|u(t^i,x^i,y^i) - u^i\right|^2 + \left|v(t^i,x^i,y^i) - v^i\right|^2\right) + \frac{1}{N}\sum_{i=1}^{N}\left(\left|f(t^i,x^i,y^i)\right|^2 + \left|g(t^i,x^i,y^i)\right|^2\right)$

Physics-informed Neural Networks (PINN)

- PINN design
  - Incompressible flow around a circular cylinder
  - Reynolds number $Re = 100$, kinematic viscosity $\nu = 0.001$
  - $N = 5000$ (1% of the total data)
  - 9 hidden layers × 20 neurons
- PDE parameter λ can be estimated from noisy, scattered data.
- Even without any pressure training data, the learned physics enables accurate reconstruction of the full pressure field.







Correct PDE	$u_t + (uu_x + vu_y) = -p_x + 0.01(u_{xx} + u_{yy})$ $v_t + (uv_x + vv_y) = -p_y + 0.01(v_{xx} + v_{yy})$
Identified PDE (clean data)	$u_t + 0.999(uu_x + vu_y) = -p_x + 0.01047(u_{xx} + u_{yy})$ $v_t + 0.999(uv_x + vv_y) = -p_y + 0.01047(v_{xx} + v_{yy})$
Identified PDE (1% noise)	$u_t + 0.998(uu_x + vu_y) = -p_x + 0.01057(u_{xx} + u_{yy})$ $v_t + 0.998(uv_x + vv_y) = -p_y + 0.01057(v_{xx} + v_{yy})$

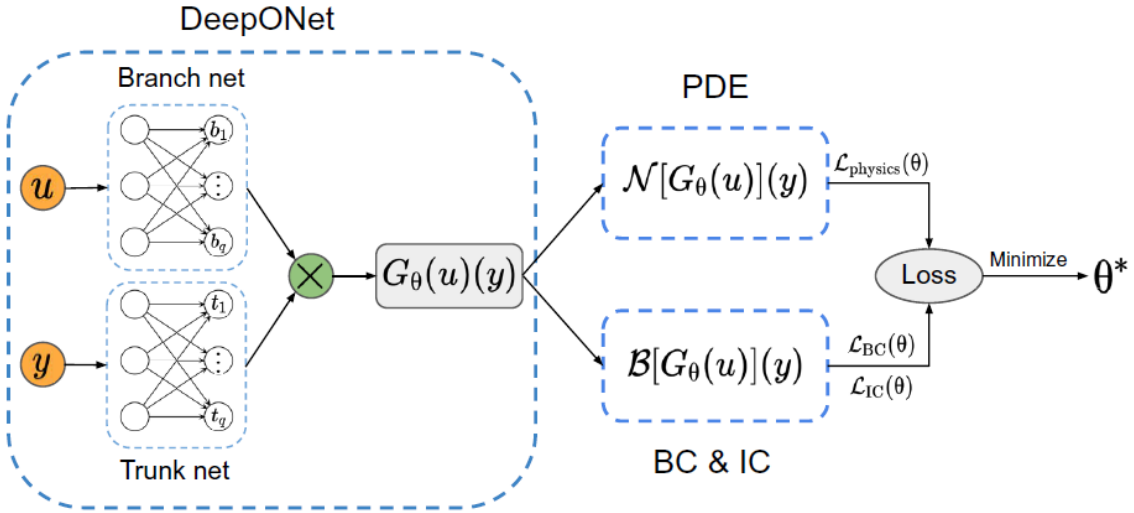
Physics-informed DeepONet (PIDON)

- Neural operator
  - It approximates the mapping $\mathcal{G}: \mathcal{A} \rightarrow \mathcal{U}$ that returns an output function $u(x)$ given an input function $a(x)$.
  - The model can be queried at any arbitrary input location (resolution independent).
  - Require large training datasets.
- DeepONet
  - A structure that separates the input function and output coordinates, representing the operator via a basis expansion.
  - $G_\theta(\mathbf{u})(\mathbf{y}) = \sum_{k=1}^{q} \underbrace{b_k(\mathbf{u}(\mathbf{x}_1), \mathbf{u}(\mathbf{x}_2) \ldots, \mathbf{u}(\mathbf{x}_m))}_{branch} \underbrace{t_k(\mathbf{y})}_{trunk}$
  - Branch network extracts latent representations of input functions.
  - Trunk network extracts latent representations of input coordinates at which the output functions are evaluated.
  - Merging the latent represnetations via a dot product.

Physics-informed DeepONet (PIDON)

- Solution operator G of a parametric PDE $\mathcal{N}(\mathbf{u}, \mathbf{s}) = 0$, $\mathcal{N}: \mathcal{U} \times \mathcal{S} \to \mathcal{V}$
  - $G: \mathcal{U} \to \mathcal{S}$, $G(\mathbf{u}) = \mathbf{s}(\mathbf{u})$
  - $\mathbf{u} \in \mathcal{U}$ denotes the input functions and $\mathbf{s} \in \mathcal{S}$ denotes the corresponding unknown solutions of the PDE system.
- Represent the solution map G by a DeepONet G_θ
  - Branch net input represents a function $\mathbf{u}$ evaluated at a collection of fixed locations $\{\mathbf{x}_i\}_{i=1}^m$.
  - Trunk net takes the continuous coordinated $\mathbf{y}$ as inputs.
- Loss function
  - $\mathcal{L} = \mathcal{L}_{operator}(\theta) + \mathcal{L}_{physics}(\theta)$

$$= \frac{1}{NP}\sum_{i=1}^{N}\sum_{j=1}^{P}\left|G_\theta\left(\mathbf{u}^{(i)}\right)\left(\mathbf{y}_{u,j}^{(i)}\right) - G\left(\mathbf{u}^{(i)}\right)\left(\mathbf{y}_{u,j}^{(i)}\right)\right|^2 + \frac{1}{NQm}\sum_{i=1}^{N}\sum_{j=1}^{Q}\sum_{k=1}^{m}\left|\mathcal{N}\left(\mathbf{u}^{(i)}(\mathbf{x}_k), G_\theta\left(\mathbf{u}^{(i)}\right)\left(\mathbf{y}_{u,j}^{(i)}\right)\right)\right|^2$$



[3] Wang, Sifan, Hanwen Wang, and Paris Perdikaris. "Learning the solution operator of parametric partial differential equations with physics-informed DeepONets." Science advances 7.40 (2021): eabi8605.

Physics-informed DeepONet (PIDON)

- Proposed models are trained without any paired input-output data
  - Assuming only knowledge of the governing equation and correponding IBCs.
  - A continuous operator surrogate that can provide output function values at arbitrary y.
- Test error $:= \frac{1}{N}\sum_{i=1}^{N} \frac{\|G_\theta(\mathbf{u}^{(i)})(\mathbf{y}) - G(\mathbf{u}^{(i)})(\mathbf{y})\|_2}{\|G(\mathbf{u}^{(i)})(\mathbf{y})\|_2}$
  - An empirical metric that evaluates how well the operator action matches over a set of test functions.
  - It does not compare the operators themselves in closed form.

$$[\mathbf{u}, \mathbf{y}, G(\mathbf{u})(\mathbf{y})] = \left[\begin{bmatrix} \vdots \\ \mathbf{u}^{(i)}(\mathbf{x}_1), \mathbf{u}^{(i)}(\mathbf{x}_2), \cdots, \mathbf{u}^{(i)}(\mathbf{x}_m) \\ \mathbf{u}^{(i)}(\mathbf{x}_1), \mathbf{u}^{(i)}(\mathbf{x}_2), \cdots, \mathbf{u}^{(i)}(\mathbf{x}_m) \\ \vdots \\ \mathbf{u}^{(i)}(\mathbf{x}_1), \mathbf{u}^{(i)}(\mathbf{x}_2), \cdots, \mathbf{u}^{(i)}(\mathbf{x}_m) \\ \vdots \end{bmatrix}, \begin{bmatrix} \vdots \\ \mathbf{y}_1^{(i)} \\ \mathbf{y}_2^{(i)} \\ \vdots \\ \mathbf{y}_P^{(i)} \\ \vdots \end{bmatrix}, \begin{bmatrix} \vdots \\ G(\mathbf{u}^{(i)})(\mathbf{y}_1^{(i)}) \\ G(\mathbf{u}^{(i)})(\mathbf{y}_2^{(i)}) \\ \vdots \\ G(\mathbf{u}^{(i)})(\mathbf{y}_P^{(i)}) \\ \vdots \end{bmatrix} \right]$$

Governing law	Equation form	Random input	Test error
Linear ODE	$\frac{ds(x)}{dx} = u(x)$	Forcing terms	$0.33 \pm 0.32\%$
Diffusion reaction	$\frac{\partial s}{\partial t} = D\frac{\partial^2 s}{\partial x^2} + ks^2 + u(x)$	Source terms	$0.45 \pm 0.16\%$
Burgers'	$\frac{\partial s}{\partial t} + s\frac{\partial s}{\partial x} - \nu\frac{\partial^2 s}{x^2} = 0$	Initial conditions	$1.38 \pm 1.64\%$
Advection	$\frac{\partial s}{\partial t} + u\frac{\partial s}{\partial x} = 0$	Variable coefficients	$2.24 \pm 0.68\%$
Eikonal	$\|\nabla s\|_2 = 0$	Domain geometries	$0.42 \pm 0.11\%$

Physics-informed DeepONet (PIDON)

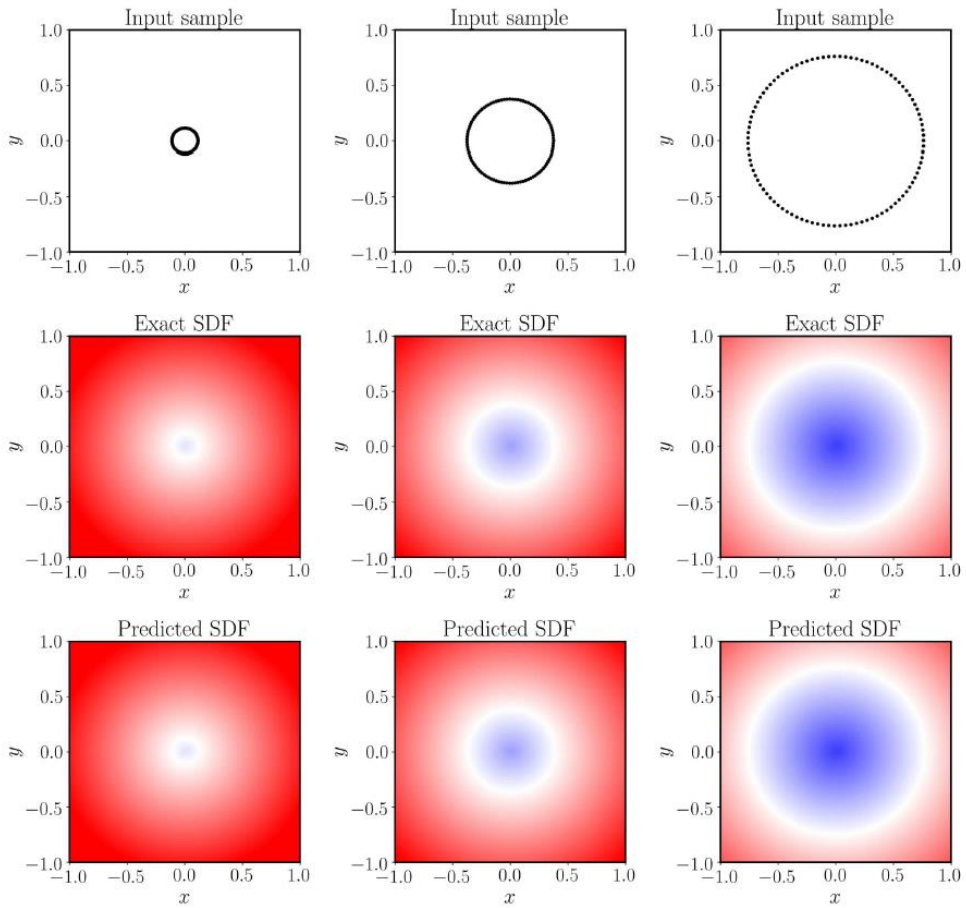
- Solving a 2D Eikonal equation $\|\nabla s(\mathbf{x})\|_2 = 1,\ s(\mathbf{x}) = 0,\ \mathbf{x} \in \partial\mathbf{\Omega}$
  - A PDE that appears in distance functions, wavefront propagation, and level-set geometry representations.
  - $s(\mathbf{x})$ acts as the signed distance function (SDF) from the boundary $\partial\mathbf{\Omega}$.
    - $s(\mathbf{x}) = \begin{cases} d(\mathbf{x}, \partial\mathbf{\Omega}) & \text{if } \mathbf{x} \in \mathbf{\Omega} \\ -d(\mathbf{x}, \partial\mathbf{\Omega}) & \text{if } \mathbf{x} \in \mathbf{\Omega}^c \end{cases}$ where $d(\mathbf{x}, \partial\mathbf{\Omega}) := \inf_{\mathbf{y} \in \partial\mathbf{\Omega}} d(\mathbf{x}, \mathbf{y})$

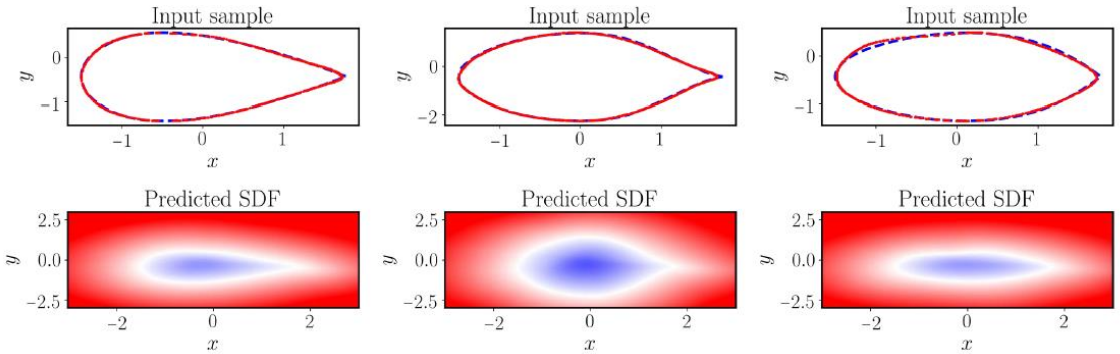
- PIDON design

$\mathbf{\Gamma}^{(i)}$ denotes a parametrized curve evaluated at a set of fixed sensor locations

  - $G: \partial\mathbf{\Omega} \mapsto s(x, y)$
  - PDE residual $R_\theta^{(i)}(x, y) = \left\|\nabla G_\theta\left(\mathbf{\Gamma}^{(i)}\right)(x, y)\right\|_2 = \left\|\sqrt{\left(\frac{dG_\theta\left(\mathbf{\Gamma}^{(i)}\right)(x,y)}{dx}\right)^2 + \left(\frac{dG_\theta\left(\mathbf{\Gamma}^{(i)}\right)(x,y)}{dy}\right)^2}\right\|_2$
  - Loss function $\mathcal{L}(\theta) = \mathcal{L}_{BC}(\theta) + \mathcal{L}_{physics}(\theta) = \frac{1}{Nm}\sum_{i=1}^{N}\sum_{j=1}^{m}\left|G_\theta\left(\mathbf{\Gamma}^{(i)}\right)(x, y)\right|^2 + \frac{1}{NQ}\sum_{i=1}^{N}\sum_{j=1}^{Q}\left|R_\theta^{(i)}\left(x_{r,j}^{(i)}, y_{r,j}^{(i)}\right) - 1\right|^2$
  - Branch and trunk net are two separate 6-layers × 100 neurons with ELU activation

Physics-informed DeepONet (PIDON)

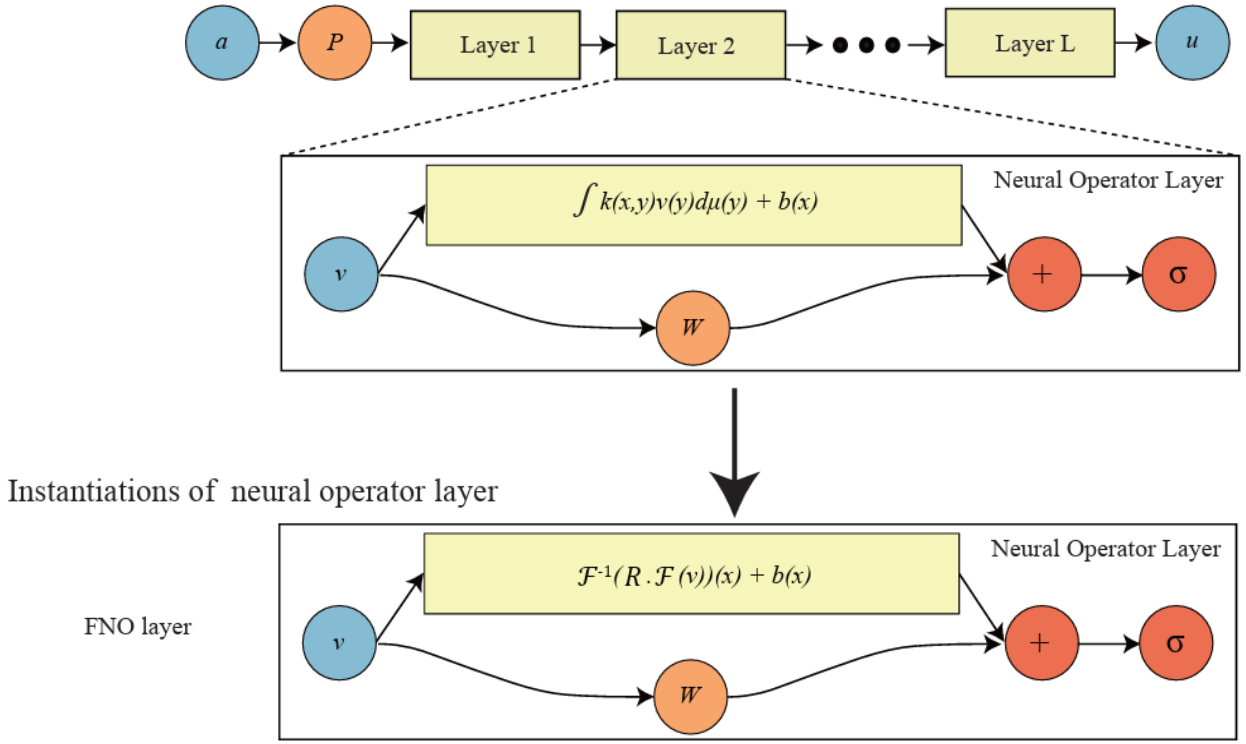




Case	Model	Test error
ODE	Physics-informed DeepONet	$0.33\% \pm 0.32\%$
	Parameterized PINN	$5.25\% \pm 5.72\%$
Diffusion-reaction	Physics-informed DeepONet	$0.25\% \pm 0.12\%$
	Parameterized PINN	$1.06\% \pm 0.74\%$
Eikonal equation (circles)	Physics-informed DeepONet	$0.42\% \pm 0.11\%$
	Parameterized PINN	$27.45\% \pm 24.12\%$

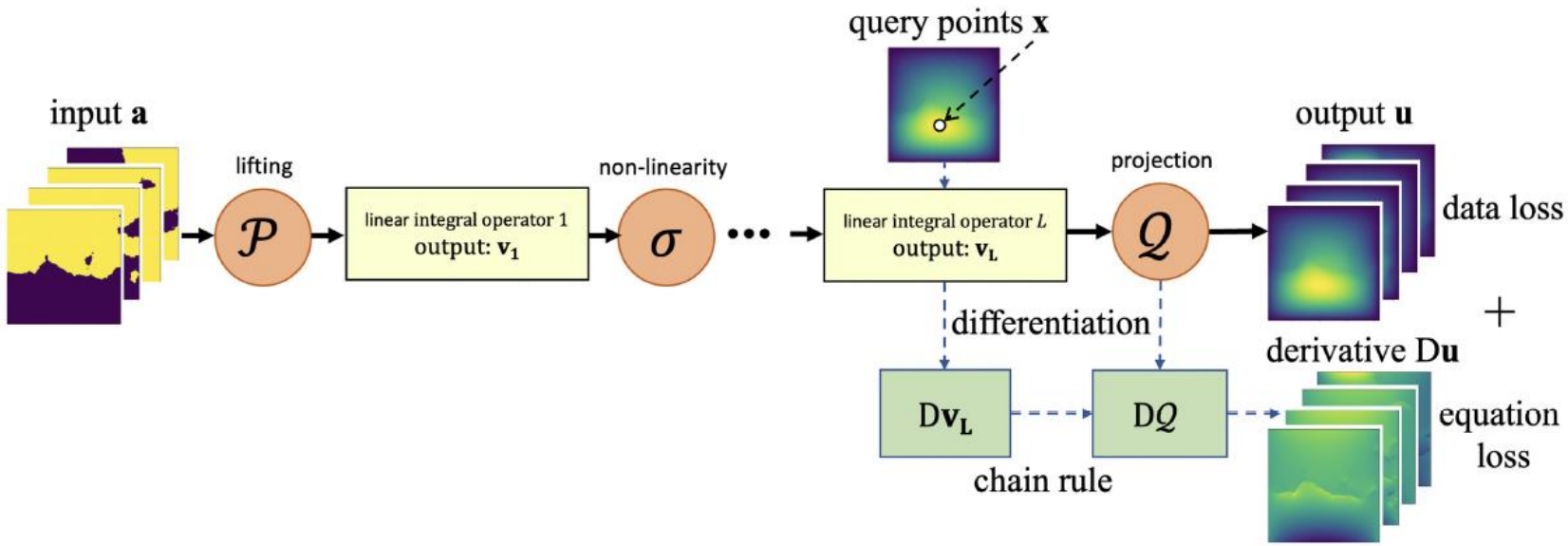
Physics-informed Neural Operator (PINO)

- Operator learning
  - $v_{t+1}(x) = \sigma\left(W v_t(x) + \int \kappa(x,y)\, v_t(y) dy\right)$
  - Layer-wise integral kernel architecture
- Fourier Neural Operator (FNO)
  - $v_{l+1}(x) = \sigma(W v_l(x) + \mathcal{F}^{-1}(R \cdot \mathcal{F}(v_l))(x))$
  - Leverages the fact that convolution-like operators become multiplications under the Fourier transform.
  - Enabling efficient implementation of global integral operations in the frequency domain.



Physics-informed Neural Operator (PINO)

- NOs can be evaluated at any data resolution at inference time without the need for retraining.
  - But, they cannot perfectly approximate the ground-truth operator when only low resolution training data is provided.
- Limitations of existing PIML methods
  - PINNs require no data but are prone to failure and generalize poorly, requiring re-optimization for new PDE instances.
  - PIDON is restricted to fixed-grid inputs and relies on a linear approximation architecture.



[4] Li, Zongyi, et al. "Physics-informed neural operator for learning partial differential equations." ACM/IMS Journal of Data Science 1.3 (2024): 1-27.

Physics-informed Neural Operator (PINO)

- Stationary system
  - $\mathcal{P}(u,a)=0,\ in\ D\subset\mathbb{R}^d;\ \ u=g,\ in\ \partial D$
  - Solution operator $\mathcal{G}^\dagger:\mathcal{A}\to\mathcal{U}$
  - PDE loss $\mathcal{L}_{\text{pde}}(a,u_\theta)=\left\|\mathcal{P}(a,u_\theta)\right\|^2_{L^2(D)}+\alpha\left\|u_\theta|_{\partial D}-g\right\|^2_{L^2(\partial D)}$

$$=\int_D|\mathcal{P}(u_\theta(x),a(x))|^2\mathrm{d}x+\alpha\int_{\partial D}|u_\theta(x)-g(x)|^2\mathrm{d}x.$$

- Dynamical system
  - $\partial u/\partial t=\mathcal{R}(u),\ in\ D\times(0,\infty);\ u=g,\ in\ \partial D\times(0,\infty);\ u=a,\ in\ \overline{D}\times\{0\}$
  - Solution operator $\mathcal{G}^\dagger:\mathcal{A}\to C((0,T];\mathcal{U})$
  - PDE loss $\mathcal{L}_{\text{pde}}(a,u_\theta)=\left\|\frac{du_\theta}{dt}-\mathcal{R}(u_\theta)\right\|^2_{L^2(T;D)}+\alpha\left\|u_\theta|_{\partial D}-g\right\|^2_{L^2(T;\partial D)}+\beta\left\|u_\theta|_{t=0}-a\right\|^2_{L^2(D)}$

$$=\int_0^T\int_D|\frac{du_\theta}{dt}(t,x)-\mathcal{R}(u_\theta)(t,x)|^2\mathrm{d}x\mathrm{d}t+\alpha\int_0^T\int_{\partial D}|u_\theta(t,x)-g(t,x)|^2\mathrm{d}x\mathrm{d}t+\beta\int_D|u_\theta(0,x)-a(x)|^2\mathrm{d}x$$

Physics-informed Neural Operator (PINO)

- Loss function to learn the solution operator $\mathcal{G}^\dagger$
  - Empirical data loss $\mathcal{L}_{data}(u, \mathcal{G}_\theta(a)) = \|u - \mathcal{G}_\theta\|_{\mathcal{U}}^2 = \int_D |u(x) - \mathcal{G}_\theta(a)(x)|^2 \, dx$
  - Operator data loss $\mathcal{J}_{data}(\mathcal{G}_\theta) = \left\|\mathcal{G}^\dagger - \mathcal{G}_\theta\right\|_{L_\mu^2(\mathcal{A};\mathcal{U})}^2 = \mathbb{E}_{a\sim\mu}[\mathcal{L}_{data}(a, \theta)] \approx \frac{1}{N}\sum_{j=1}^N \int_D |u(x) - \mathcal{G}_\theta(a)(x)|^2 \, dx$
  - Operator PDE loss $\mathcal{J}_{pde}(\mathcal{G}_\theta) = \mathbb{E}_{a\sim\mu}[\mathcal{L}_{pde}(a, \mathcal{G}_\theta(a))]$
- 2-phase PINO framework
  1. Operator learning: learn $\mathcal{G}_\theta$ to approximate $\mathcal{G}^\dagger$ using either/both $\mathcal{J}_{data}$ or/and $\mathcal{J}_{pde}$.
  2. Instance-wise fine-tuning: use $\mathcal{G}_\theta(a)$ as the ansatz to approximate $u^\dagger$ with $\mathcal{L}_{pde}$ and/or $\mathcal{L}_{op}$
     - Optional anchor loss $\mathcal{L}_{op}(\mathcal{G}_{\theta_i}(a), \mathcal{G}_{\theta_0}(a)) \coloneqq \left\|\mathcal{G}_{\theta_i}(a) - \mathcal{G}_{\theta_0}(a)\right\|_{\mathcal{U}}^2$

Physics-informed Neural Operator (PINO)

- Derivatives of neural operators
  - In general, it is non-trivial to compute the derivatives $d\mathcal{G}_\theta(a)/dx$ and $d\mathcal{G}_\theta(a)/dt$ for model $\mathcal{G}_\theta$.
  - $\mathcal{G}_\theta \coloneqq \mathcal{Q} \circ (\mathcal{W}_L + \mathcal{K}_L) \circ \cdots \circ \sigma(\mathcal{W}_1 + \mathcal{K}_1) \circ \mathcal{P}$
  - $u(x) = \mathcal{G}_\theta(a)(x) = \mathcal{Q}\big(v_L(x)\big) = \mathcal{Q}\big((\mathcal{W}_L v_{L-1})(x) + \mathcal{K}_L v_{L-1}(x)\big)$
  - Function-wise differentiation $u' = \mathcal{Q}'(v_L) \cdot \mathcal{F}^{-1}\left(\frac{i2\pi}{D} K \cdot (\mathcal{F} v_L)\right)$
    - Derivatives are computed in batch and explicitly expressed in Fourier space with the chain rule..
- Inverse problem
  - Learn the inverse operator $\mathcal{F}_\theta : u \longmapsto a$ and use $\mathcal{F}_\theta(u^\dagger)$ to approxiamte $a^\dagger$.
  - Inverse loss function $\mathcal{J}_{backward} \coloneqq \mathcal{L}_{pde}(\mathcal{F}_\theta(u^\dagger), u^\dagger) + \mathcal{L}_{op}\left(\mathcal{F}_\theta(u^\dagger), \mathcal{F}_{\theta_0}(u^\dagger)\right) + R\left(\mathcal{F}_\theta(u^\dagger)\right)$

Physics-informed Neural Operator (PINO)

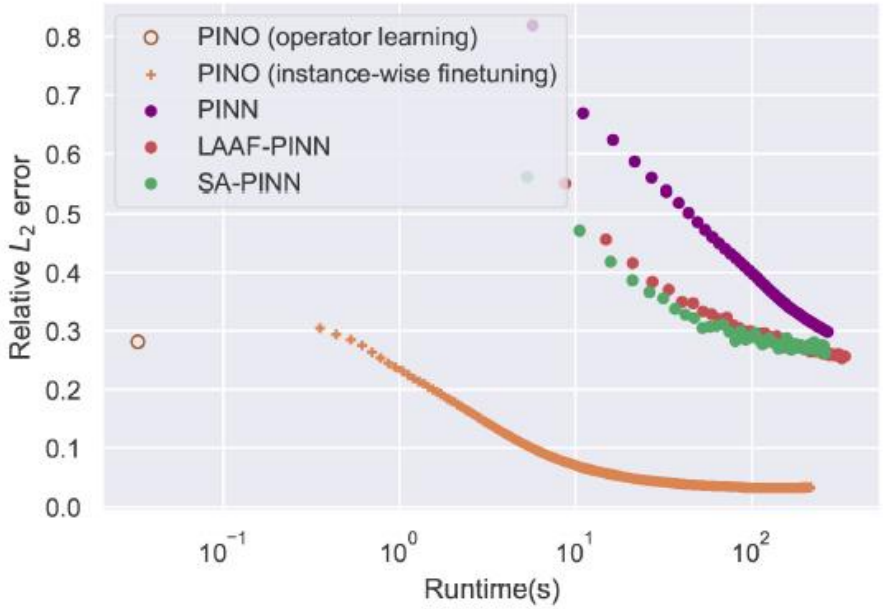
Table 1. Operator-Learning Using Fixed Resolution Data and PDE Loss Allows us to Train Operators with High Accuracy on Higher Resolution Unseen Data

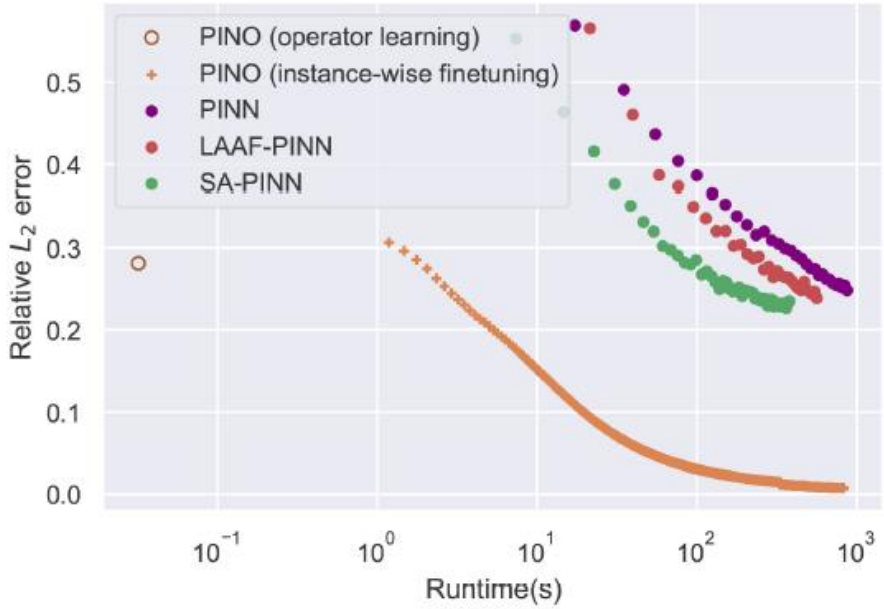
PDE	Training setting	Error at low data resolution	Error at 2× data resolution	Error at 4× data resolution
Burgers	Data	0.32±0.01%	3.32±0.02%	3.76±0.02%
	Data and PDE loss	0.17±0.01%	0.28±0.01%	0.38±0.01%
Darcy	Data	5.41±0.12%	9.01±0.07%	9.46±0.07%
	Data and PDE loss	5.23±0.12%	1.56±0.05%	1.58±0.06%
Kolmogorov flow	Data	8.28%±0.15%	8.27%±0.15%	8.30%±0.15%
	Data and PDE loss	6.04%±0.12%	6.02%±0.12%	6.01%±0.12%

Table 2. Operator Learning on Darcy Flow Equation

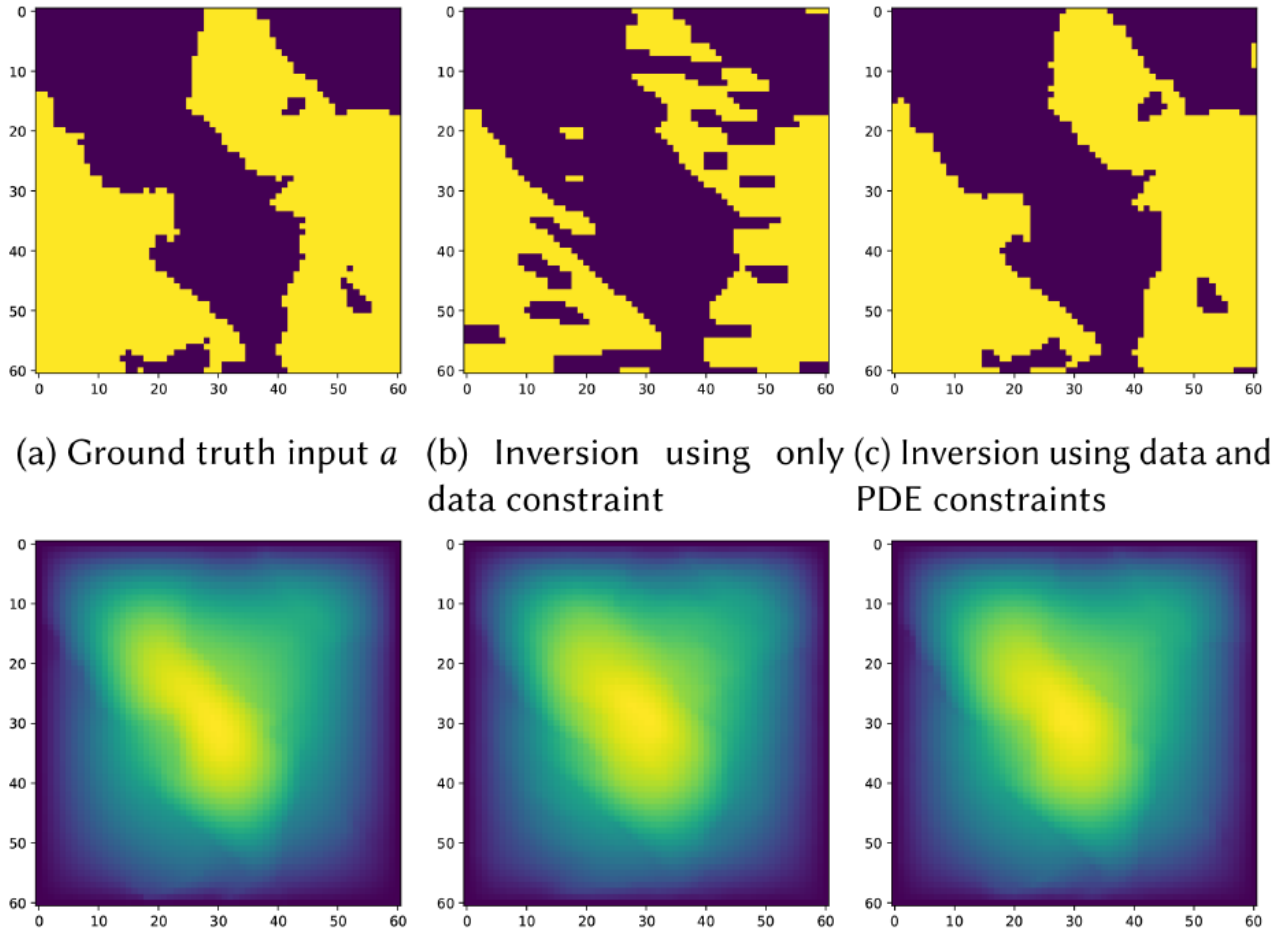
Method	Solution error
DeepONet with data [4]	6.97 ± 0.09%
PINO with data	1.22 ± 0.03%
PINO w/o data	1.50 ± 0.03%

Incorporating physics constraints in operator learning improves the performance of neural operators.





Physics-informed Neural Operator (PINO)



(a) Ground truth input a (b) Inversion using only data constraint (c) Inversion using data and PDE constraints

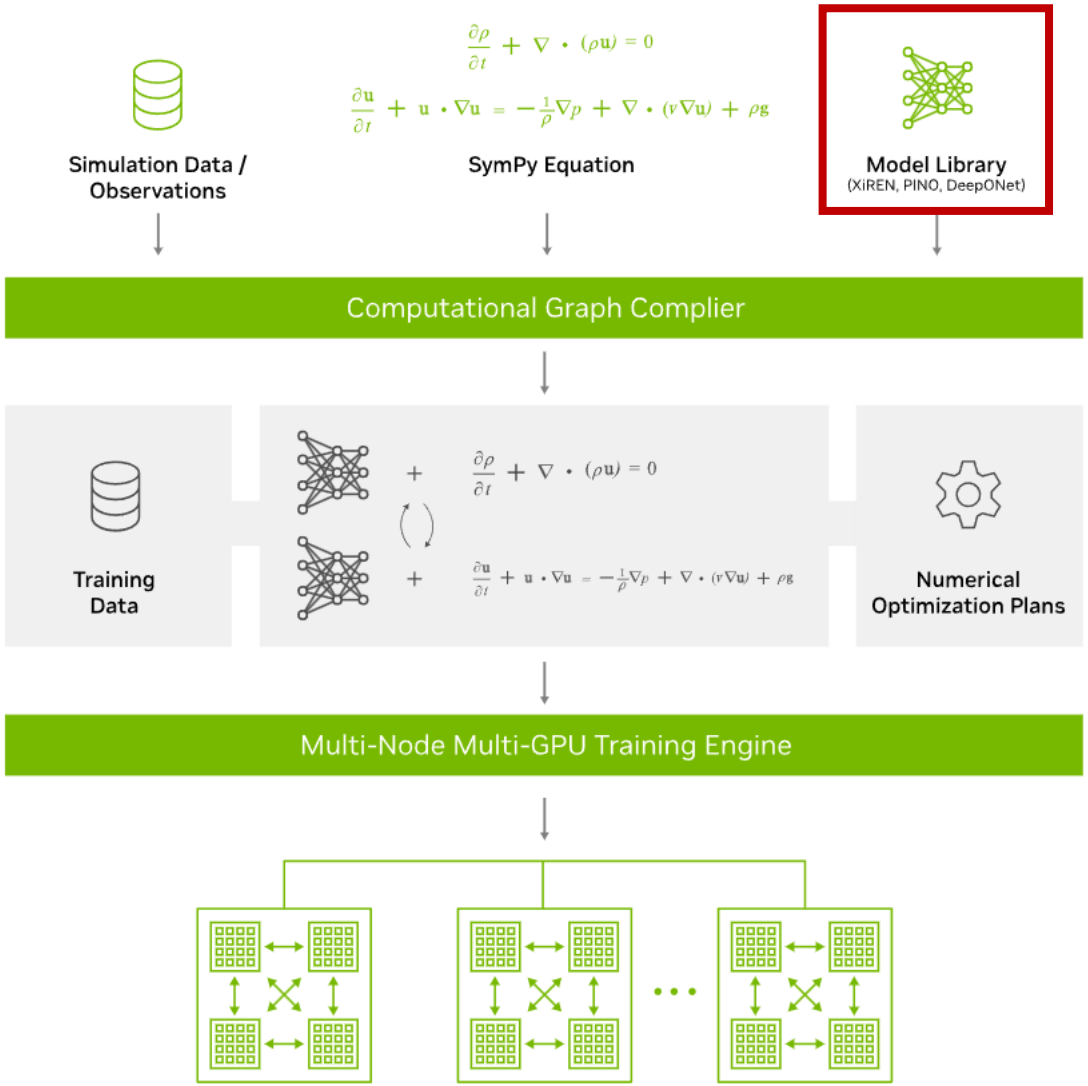
(d) Observed output function (e) Output function of inversion using only data constraint (f) Output function of inversion using data and PDE constraints

Forward problem

Inverse problem

Related Works

- Science/Engineering applications
  - Quantum mechanics
  - Electromagnetic channel estimaton
- NVIDIA PhysicsNeMo
  - Open-source Python framework for building, training, and fine-tuning physics AI models at scale.
  - SK Hyinx has developed an AI surrogate model for etching process simulation, accelerating semiconductor development.
  - LG Display has developed a Digital Panel Solution (DPS), a digital twin tool for OLED panel development.



Conclusion

- PIML is a mature field, with demonstrated performance across diverse applications.
- A variety of PIML architectures exist and can be readily applied using open-source frameworks.
- Applications in wireless communications are increasingly showing promising results.

- Future works
  - Introduce PIML into HMIMO systems by learning the Helmholtz or Maxwell equations.
  - Introduce PIML into quantum communication systems by learning quantum theory.

References

- [1] Karniadakis, George Em, et al. "Physics-informed machine learning." Nature Reviews Physics 3.6 (2021): 422-440.
- [2] Raissi, Maziar, Paris Perdikaris, and George E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations." Journal of Computational physics 378 (2019): 686-707.
- [3] Wang, Sifan, Hanwen Wang, and Paris Perdikaris. "Learning the solution operator of parametric partial differential equations with physics-informed DeepONets." Science advances 7.40 (2021): eabi8605.
- [4] Li, Zongyi, et al. "Physics-informed neural operator for learning partial differential equations." ACM/IMS Journal of Data Science 1.3 (2024): 1-27.